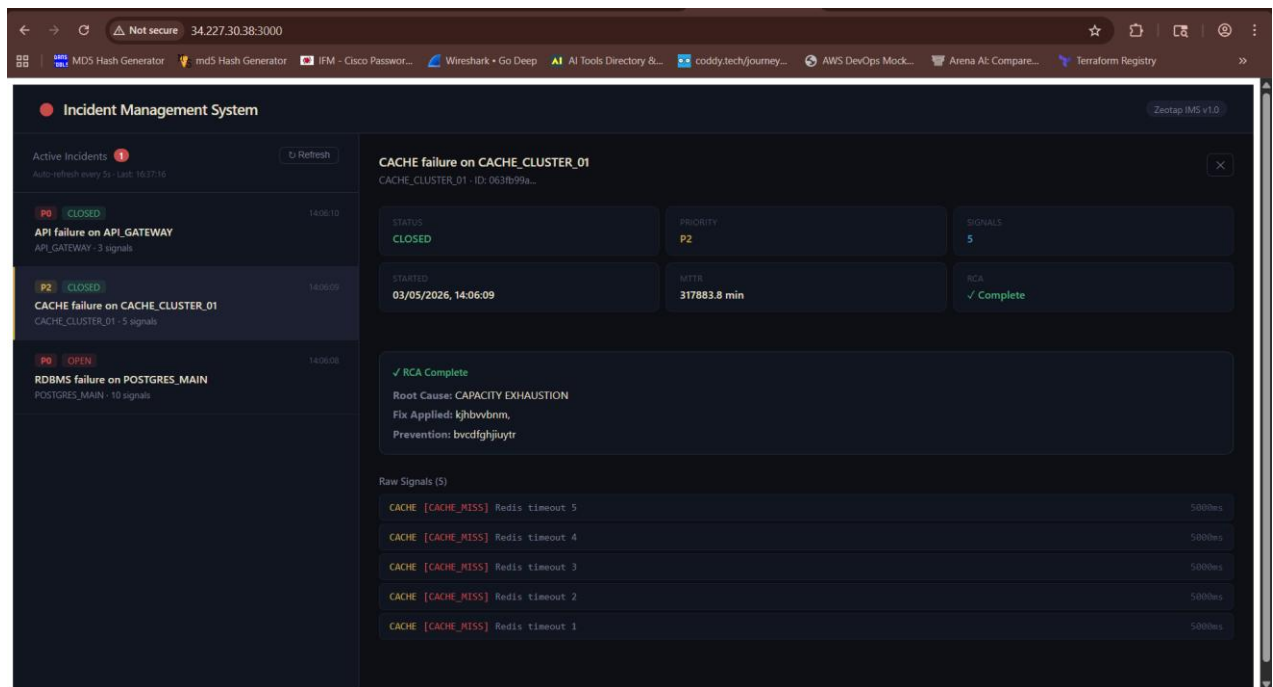


## MHA Ramayya – Infrastructure / SRE Intern Assignment

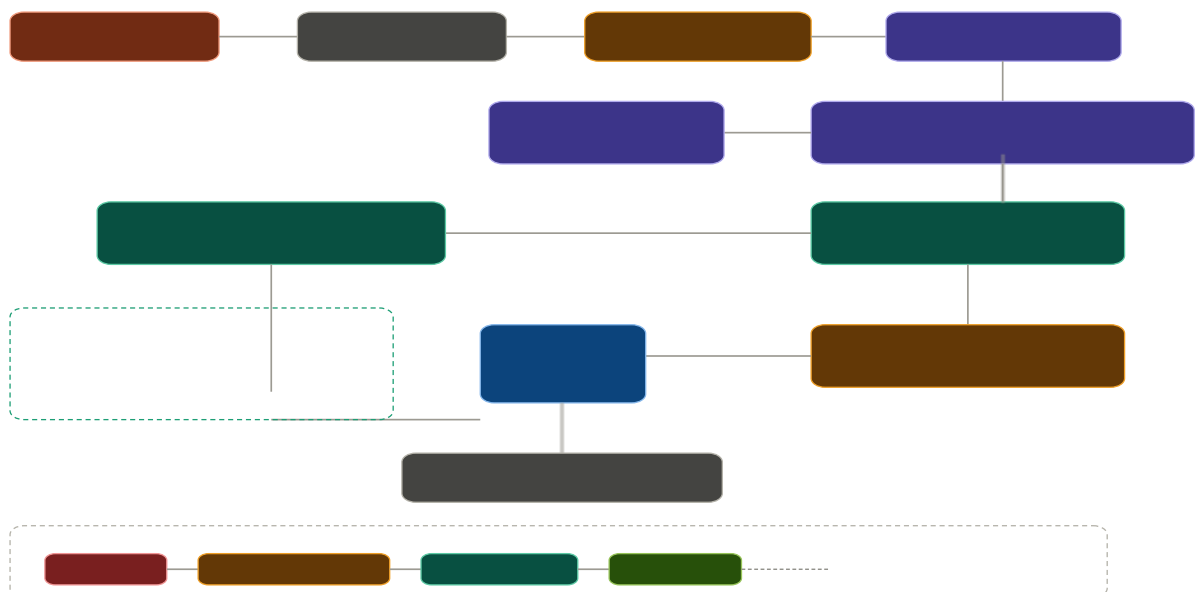


### 1. Project Overview

This project implements a **production-style Incident Management System (IMS)** with a complete DevOps lifecycle: CI/CD, monitoring, observability, and containerization. It demonstrates automation, resilience, and scalable deployment practices.

2. GitHub Repository <https://github.com/babai-cyber/ims.git>

### 3. Architecture Diagram



**Frontend (React + Vite) → Backend (Node.js + TypeScript + Express) → PostgreSQL (Work Items + RCA) + MongoDB (Raw Signals) + Redis (Cache + Queue)** All services are containerized with **Docker Compose** and orchestrated via **Jenkins CI/CD**. Monitoring is powered by **Prometheus + Grafana**.

#### 4. Architecture Explanation

- **Frontend:** React dashboard auto-refreshing every 5s, showing incidents and RCA forms.
- **Backend:** Node.js API with Bull queue for backpressure, Redis debounce logic, PostgreSQL for transactional state, MongoDB for raw signals.
- **Databases:** PostgreSQL (ACID transactions), MongoDB (TTL audit log), Redis (cache + queue).
- **Design Patterns:**
  - Strategy Pattern → P0 alerts for RDBMS/API, P2 alerts for Cache/Queue.
  - State Machine → Enforces OPEN → INVESTIGATING → RESOLVED → CLOSED lifecycle.

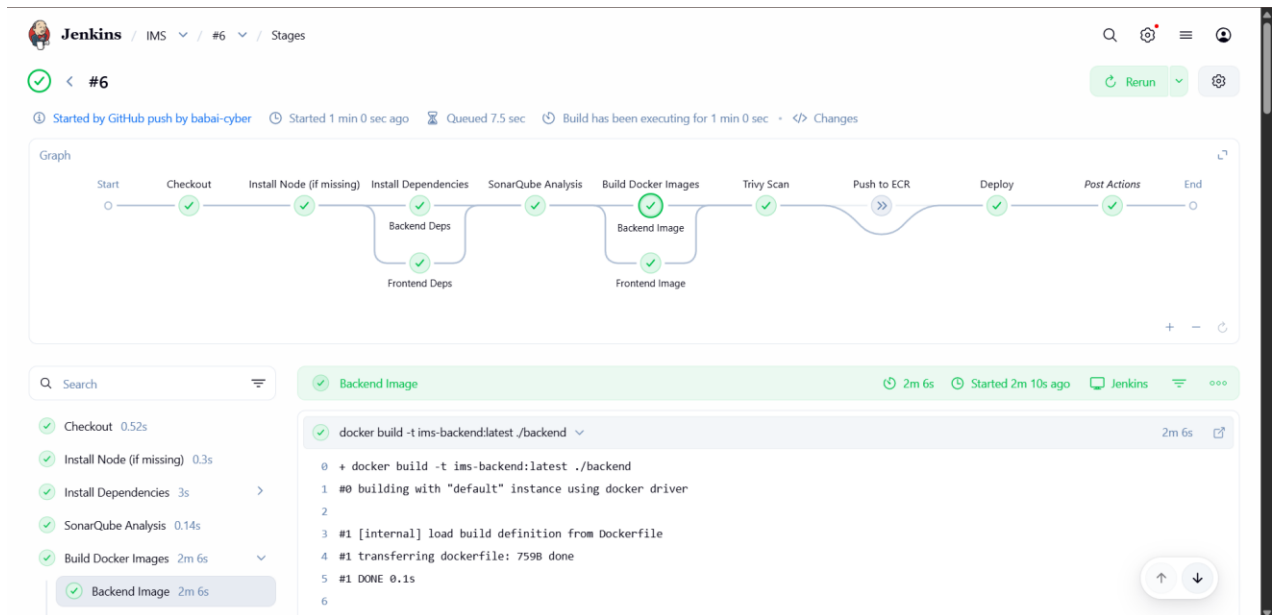
#### 5. CI/CD Pipeline (Jenkins)

**Before webhook trigger :**

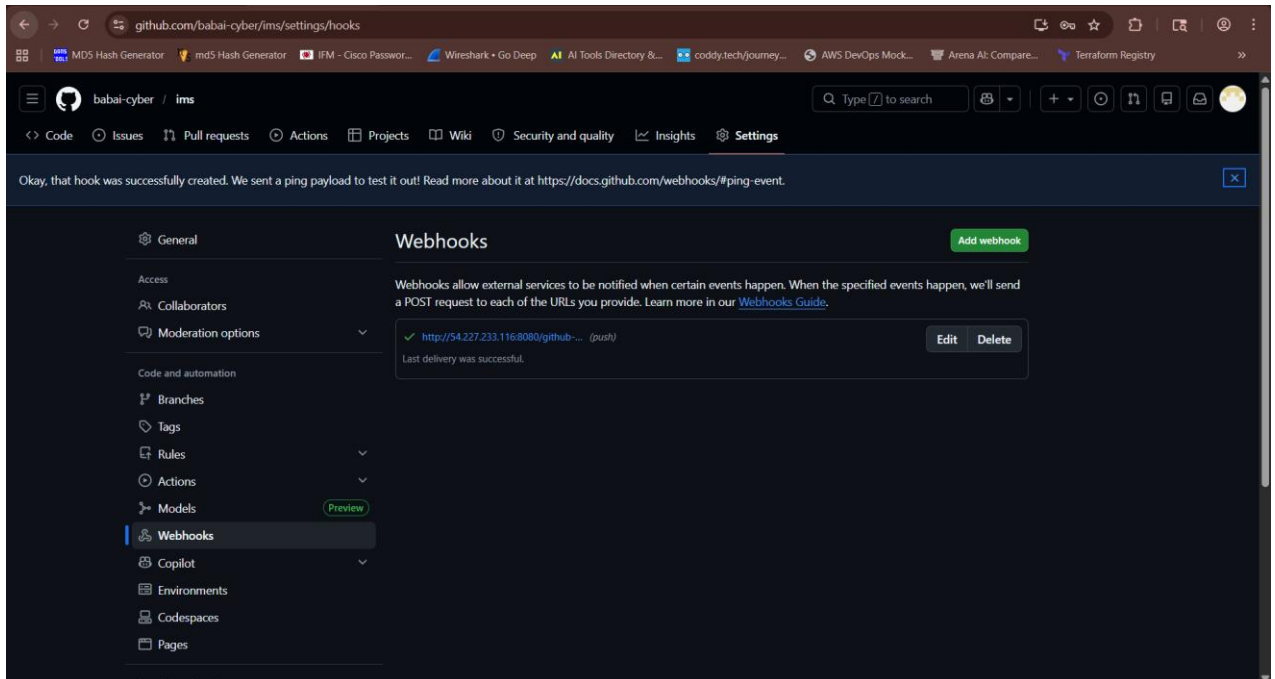
The screenshot shows the Jenkins interface for a pipeline named 'IMS'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Favorite, Open Blue Ocean, Stages, Rename, and Pipeline Syntax. The main area displays a table of build history with columns for stages and their durations. The stages are: Checkout, Install Node (if missing), Install Dependencies, Backend Deps, Frontend Deps, SonarQube Analysis, Build Docker Images, Backend Image, Frontend Image, Trivy Scan, Push to ECR, Deploy, and Declarat Post Actor.

| Checkout | Install Node (if missing) | Install Dependencies | Backend Deps | Frontend Deps | SonarQube Analysis | Build Docker Images | Backend Image | Frontend Image | Trivy Scan  | Push to ECR | Deploy       | Declarat Post Actor |
|----------|---------------------------|----------------------|--------------|---------------|--------------------|---------------------|---------------|----------------|-------------|-------------|--------------|---------------------|
| 419ms    | 338ms                     | 93ms                 | 3s           | 1s            | 195ms              | 84ms                | 3s            | 4s             | 270ms       | 83ms        | 171ms        | 2s                  |
| 635ms    | 363ms                     | 117ms                | 3s           | 1s            | 238ms              | 88ms                | 10s           | 11s            | 660ms       |             | 333ms        | 6s                  |
| 403ms    | 324ms                     | 82ms                 | 2s           | 1s            | 158ms              | 82ms                | 396ms failed  | 450ms failed   | 71ms failed | 70ms failed | 71ms failed  | 361m                |
| 221ms    | 328ms                     | 82ms                 | 2s           | 1s            | 190ms              | 83ms                | 649ms failed  | 705ms failed   | 81ms failed | 96ms failed | 109ms failed | 367m                |

Below the table, there are 'Permalinks' for the last build (#5), last stable build (#5), and last successful build (#5).



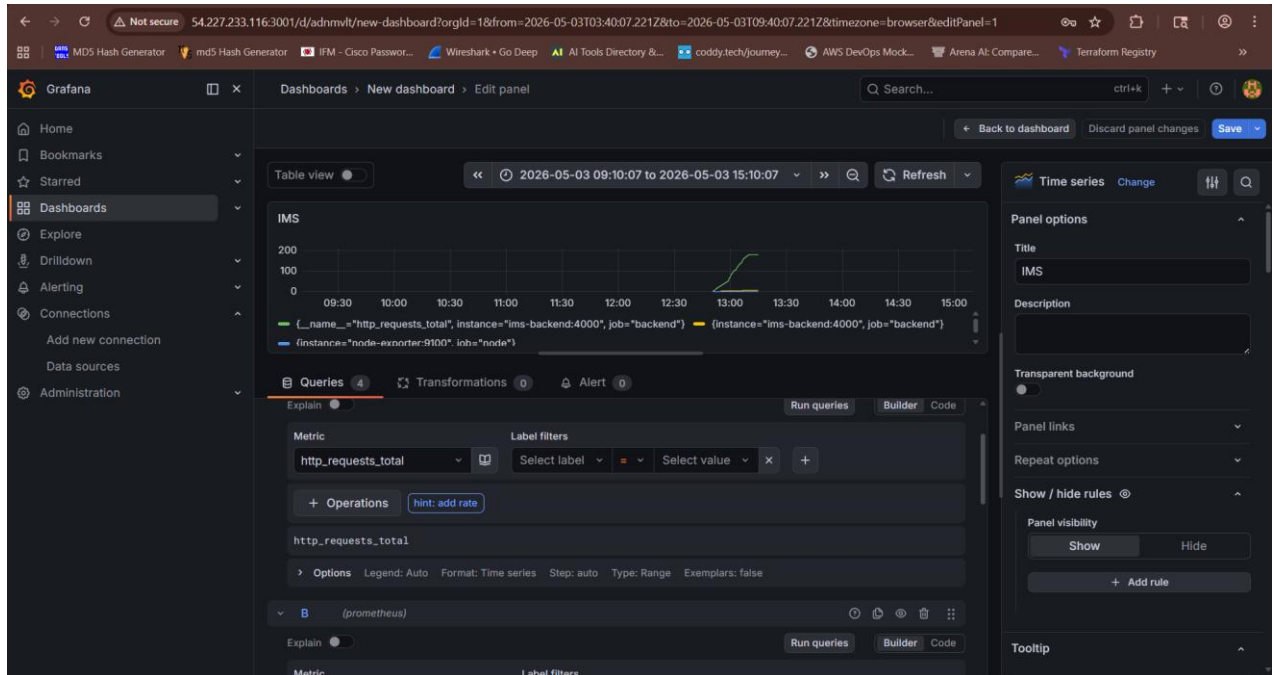
After Webhook trigger:



Stages:

1. **Checkout** (GitHub webhook) : <https://github.com/babai-cyber/ims.git>
2. **Install Dependencies** (npm ci)
3. **Build** (TypeScript compile)
4. **Unit Tests** (Jest for RCA/state machine/debounce)
5. **SonarQube Analysis** (Quality Gate enforcement)
6. **Trivy Scan** (Docker image CVE scanning)

7. **Docker Build & Push** (Docker Hub/ECR)
8. **Deploy** (EC2 via docker-compose)
9. **Health Check** (/health endpoint)
6. **Monitoring (Prometheus + Grafana)**



- **Prometheus** scrapes metrics every 15s from backend and node-exporter.
- **Grafana Dashboard Panels:**
  - Signal Throughput (rate of signals/sec) ○ Queue Depth
  - (Bull jobs waiting) ○ HTTP Latency (P99 histogram) ○
  - Open Incidents Count

## 7. Security & Best Practices

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)

**sonarqube** Projects Issues Rules Quality Profiles Quality Gates Administration

Quality Gates [Create](#)

Sonar way **BUILT-IN** [Copy](#)

**This quality gate complies with Clean as You Code**

This quality gate complies with the [Clean as You Code](#) methodology, so that you benefit from the most efficient approach to delivering Clean Code. It ensures that:

- No new bugs are introduced
- No new vulnerabilities are introduced
- All new security hotspots are reviewed
- New code has limited technical debt
- New code has limited duplication
- New code is properly covered by tests

Conditions [Conditions](#)

Conditions on New Code

| Metric                     | Operator        | Value                                      |
|----------------------------|-----------------|--------------------------------------------|
| Coverage                   | is less than    | 80.0%                                      |
| Duplicated Lines (%)       | is greater than | 3.0%                                       |
| Maintainability Rating     | is worse than   | A (Technical debt ratio is less than 5.0%) |
| Reliability Rating         | is worse than   | A (No bugs)                                |
| Security Hotspots Reviewed | is less than    | 100%                                       |
| Security Rating            | is worse than   | A (No vulnerabilities)                     |

- **Rate Limiting:** 10k requests/min per IP.
- **Input Validation:** Required fields enforced at API level.
- **CORS + Helmet.js:** Secure headers, restricted origins.
- **Secrets Management:** Environment variables + Jenkins credentials store.
- **Trivy CVE Scanning:** Fail pipeline on CRITICAL vulnerabilities.
- **SonarQube SAST:** No blockers/critical hotspots.
- **MongoDB TTL Index:** Auto-delete signals after 90 days.
- **AWS Security Groups:** Only required ports exposed.

## 8. Deployment

The image shows two screenshots. The top screenshot is an AWS CLI terminal window displaying the output of the `docker ps` command. It lists various containers including Grafana, Prometheus, Redis, and MongoDB, along with their IDs, images, commands, and status. The bottom screenshot shows the Zeatap Incident Management System (IMS) v1.0 interface. It displays 'Active Incidents' and a message stating 'No active incidents' with a green checkmark. Below this message, it says 'Run seed/seed.sh to simulate failures'.

| CONTAINER ID      | IMAGE              | COMMAND                   | CREATED        | STATUS                           | PORTS                                               |
|-------------------|--------------------|---------------------------|----------------|----------------------------------|-----------------------------------------------------|
| 21e7c2346285      | grafana/grafana    | "/run.sh"                 | 32 seconds ago | Up 31 seconds                    | 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp         |
| 492b737de38e      | ims-frontend       | "/docker-entrypoint.s..." | 32 seconds ago | Up 31 seconds (health: starting) | 80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp |
| 01c940a4e954      | prom/prometheus    | "/bin/prometheus --c..."  | 32 seconds ago | Up 31 seconds                    | 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp         |
| 34fer5e73e01      | ims-backend        | "docker-entrypoint.s..."  | 32 seconds ago | Up 31 seconds (healthy)          | 0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp         |
| 9a0de8318ad9      | postgres:15-alpine | "docker-entrypoint.s..."  | 33 seconds ago | Up 32 seconds                    | 5432/tcp                                            |
| 40878d2927b0      | redis:7-alpine     | "docker-entrypoint.s..."  | 33 seconds ago | Up 32 seconds                    | 6379/tcp                                            |
| ims-redis         | mongo:7            | "docker-entrypoint.s..."  | 33 seconds ago | Up 32 seconds                    | 27017/tcp                                           |
| af0338436f47      | ims-mongo          | "docker-entrypoint.s..."  | 33 seconds ago | Up 32 seconds                    | 27017/tcp                                           |
| 1f1c22569417      | prom/node-exporter | "/bin/node_exporter"      | 33 seconds ago | Up 32 seconds                    | 0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp         |
| ims-node-exporter |                    |                           |                |                                  |                                                     |

- **Local:** docker-compose up --build
- **Cloud:** AWS EC2 (t3.medium) .
- **Health Check:** curl http://<ec2-ip>:4000/health
- **Frontend:** http://<ec2-ip>:3000

- **Monitoring:** Prometheus (:9090), Grafana (:3001).

## 9. Screenshots (Attached in Submission) [link for project screenshots](#)

- Application UI (Incident Dashboard)
- Jenkins Pipeline Execution
- Grafana Dashboard
- Prometheus Targets
- SonarQube Quality Gate Report

## 10. Conclusion

This project demonstrates **SRE capabilities**:

- Handling **10k signals/sec** with backpressure.
- Enforcing **debounce logic** and RCA gating.
- Full **DevSecOps pipeline** with SonarQube + Trivy.
- **Observability stack** with Prometheus + Grafana.
- Secure, resilient, and scalable deployment on Docker + AWS EC2.